



AI & ML LAB

Artificial Intelligence Lab

EXPERIMENTAL MANUAL

LIST OF EXPERIMENTS

1. A * Search Algorithm
2. Optimization Algorithm - Hill Climb
3. Optimization Algorithm - Simulated Annealing
4. Optimization Algorithm - Genetic Algorithm
5. Fuzzy Logic

EXPERIMENT NO.: 1

NAME

A * Search Algorithm

OBJECTIVE

To perform A * Search Algorithm to find the shortest route from Arad to Bucharest.

THEORY

A* search algorithm is a well-known **heuristic** search method that calculates the distance between the current state and the objective state using a heuristic function. The A* search method adds the actual cost from the start node to the current node and the predicted cost from the current node to the target node to determine the cost of each node. A heuristic function that estimates the distance between the **current node** and the desired node is used to determine the estimated cost. The algorithm then chooses the node with the **lowest cost**, grows it, and keeps doing this until it reaches the **destination node**.

As long as the heuristic function is acceptable and consistent, the A* search algorithm assures finding the **shortest path** to the destination node. This makes it an ideal search method. A heuristic function is considered acceptable if it never overestimates the destination node's distance. According to the triangle inequality, a consistent heuristic function is one in which the estimated cost from the current node to the target node is less than or equal to the actual cost plus the estimated cost from the next node to the **goal node**.

Like Dijkstra, A* works by making a lowest-cost path [tree](#) from the start node to the target node. What makes A* different and better for many searches is that for each node, A* uses a function $f(n)$ that gives an estimate of the total cost of a path using that node. Therefore, A* is a heuristic function, which differs from an algorithm in that a heuristic is more of an estimate and is not necessarily provably correct.

A* expands paths that are already less expensive by using this function:

$$f(n) = g(n) + h(n),$$

where

- $f(n)$ = total estimated cost of path through node n
- $g(n)$ = cost so far to reach node n
- $h(n)$ = estimated cost from n to goal. This is the heuristic part of the cost function, so it is like a guess.

A* Algorithm: Step-by-Step Guide

Initialization

1. Create an open list to track nodes for evaluation.

2. Make a closed list for nodes that have been evaluated.
3. Add the start node to the open list, marking the beginning of your path.

Main Loop

Continue until the open list is empty or the goal is reached:

1. Select the node with the lowest $f(x)$ value, indicating the most promising path.
2. Move the selected node from the open list to the closed list.
3. Examine each neighbor of the selected node to determine the next steps.

Evaluating Neighbors

1. For each neighbor:
 - If it is the goal, reconstruct the path and return it as the solution.
 - Skip any neighbors already in the closed list, as they have been evaluated.
2. If a neighbor is not in the open list:
 - Add it and calculate its $g(x)$, $h(x)$, and $f(x)$ values.
3. For neighbors already in the open list:
 - Check if the new path is more efficient (lower $g(x)$ value).
 - If so, update the $g(x)$, $h(x)$, and $f(x)$ values, and set the current node as its parent.

PROCEDURE

1. Create a Class Vertex.

```

  A* Star Search

  Graph

  class Vertex:

    def __init__(self, label, distance_objective):
        self.label = label
        self.visited = False
        self.distance_objective = distance_objective
        self.adjacent = []

    def add_adjacent(self, adjacent):
        self.adjacent.append(adjacent)

    def print_adjacent(self):
        for i in self.adjacent:
            print(i.vertex.label, i.cost)
```

2. Create a Class Adjacent.

```

[2] class Adjacent:
    def __init__(self, vertex, cost):
        self.vertex = vertex
        self.cost = cost
        self.astar_distance = vertex.distance_objective + self.cost
```

3. Create a Class Graph.

```

class Graph:
    arad = Vertex('Arad', 366)
    zerind = Vertex('Zerind', 374)
    oradea = Vertex('Oradea', 380)
    sibiu = Vertex('Sibiu', 253)
    timisoara = Vertex('Timisoara', 329)
    lugoj = Vertex('Lugoj', 244)
    mehadia = Vertex('Mehadia', 241)
    dobroeta = Vertex('Dobreta', 242)
    craiova = Vertex('Craiova', 160)
    rimnicu = Vertex('Rimnicu', 193)
    fagaras = Vertex('Fagaras', 178)
    pitesti = Vertex('Pitesti', 98)
    bucharest = Vertex('Bucharest', 0)
    giurgiu = Vertex('Giurgiu', 77)

    arad.add_adjacent(Adjacent(zerind, 75))
    arad.add_adjacent(Adjacent(sibiu, 140))
    arad.add_adjacent(Adjacent(timisoara, 118))

    zerind.add_adjacent(Adjacent(arad, 75))
    zerind.add_adjacent(Adjacent(oradea, 71))

    oradea.add_adjacent(Adjacent(zerind, 71))
    oradea.add_adjacent(Adjacent(sibiu, 151))

    sibiu.add_adjacent(Adjacent(oradea, 151))
    sibiu.add_adjacent(Adjacent(arad, 140))
    sibiu.add_adjacent(Adjacent(fagaras, 99))
    sibiu.add_adjacent(Adjacent(rimnicu, 80))

    timisoara.add_adjacent(Adjacent(arad, 118))
    timisoara.add_adjacent(Adjacent(lugoj, 111))

    lugoj.add_adjacent(Adjacent(timisoara, 111))
    lugoj.add_adjacent(Adjacent(mehadia, 70))

    mehadia.add_adjacent(Adjacent(lugoj, 70))
    mehadia.add_adjacent(Adjacent(dobreta, 75))

    dobroeta.add_adjacent(Adjacent(mehadia, 75))
    dobroeta.add_adjacent(Adjacent(craiova, 120))

    craiova.add_adjacent(Adjacent(dobreta, 120))
    craiova.add_adjacent(Adjacent(pitesti, 138))
    craiova.add_adjacent(Adjacent(rimnicu, 146))

    rimnicu.add_adjacent(Adjacent(craiova, 146))
    rimnicu.add_adjacent(Adjacent(sibiu, 80))
    rimnicu.add_adjacent(Adjacent(pitesti, 97))

    dobroeta.add_adjacent(Adjacent(craiova, 120))

    craiova.add_adjacent(Adjacent(dobreta, 120))
    craiova.add_adjacent(Adjacent(pitesti, 138))
    craiova.add_adjacent(Adjacent(rimnicu, 146))

    rimnicu.add_adjacent(Adjacent(craiova, 146))
    rimnicu.add_adjacent(Adjacent(sibiu, 80))
    rimnicu.add_adjacent(Adjacent(pitesti, 97))

    fagaras.add_adjacent(Adjacent(sibiu, 99))
    fagaras.add_adjacent(Adjacent(bucharest, 211))

    pitesti.add_adjacent(Adjacent(rimnicu, 97))
    pitesti.add_adjacent(Adjacent(craiova, 138))
    pitesti.add_adjacent(Adjacent(bucharest, 101))

    bucharest.add_adjacent(Adjacent(fagaras, 211))
    bucharest.add_adjacent(Adjacent(pitesti, 101))
    bucharest.add_adjacent(Adjacent(giurgiu, 90))

[4] graph = Graph()

```

4. Create a Class Ordered Array.

```

v Ordered array

[5] import numpy as np
class OrderedArray:

    def __init__(self, capacity):
        self.capacity = capacity
        self.last_position = -1
        self.values = np.empty(self.capacity, dtype=object)

    def print(self):
        if self.last_position == -1:
            print('The array is empty!')
        else:
            for i in range(self.last_position + 1):
                print(i, ' - ', self.values[i].vertex.label, ' - ',
                      self.values[i].cost, ' - ',
                      self.values[i].vertex.distance_objective, ' - ',
                      self.values[i].astar_distance)

    def insert(self, adjacent):
        if self.last_position == self.capacity - 1:
            print('Maximum capacity has been reached!')
            return

        position = 0
        for i in range(self.last_position + 1):
            position = i
            if self.values[i].astar_distance > adjacent.astar_distance:
                break
            if i == self.last_position:
                position = i + 1

        x = self.last_position
        while x >= position:
            self.values[x + 1] = self.values[x]
            x -= 1

        self.values[x + 1] = adjacent
        self.last_position += 1

[6] graph.arad.adjacent

[<__main__.Adjacent at 0x77fee0101c00>,
 <__main__.Adjacent at 0x77fee0101c60>,
 <__main__.Adjacent at 0x77fee0101cc0>]

graph.arad.adjacent[0].vertex.label, graph.arad.adjacent[0].vertex.distance_objective
('Zerind', 374)

[8] graph.arad.adjacent[0].astar_distance, graph.arad.adjacent[0].cost
(449, 75)

[9] 374 + 75
449

[9] 374 + 75
449

array = OrderedArray(20)
array.insert(graph.arad.adjacent[0])
array.insert(graph.arad.adjacent[1])
array.insert(graph.arad.adjacent[2])

array.print()
0 - Sibiu - 140 - 253 - 393
1 - Timisoara - 118 - 329 - 447
2 - Zerind - 75 - 374 - 449

```

OUTPUT

1. Create a Class AStar.

EXPERIMENT NO.: 2

NAME

Hill Climb Optimization Algorithm

OBJECTIVE

To perform Hill Climb Optimization Algorithm to find the cheapest flights between the source destinations and Rome.

THEORY

Hill climbing is a simple optimization algorithm used in Artificial Intelligence (AI) to find the best possible solution for a given problem. It belongs to the family of local search algorithms and is often used in optimization problems where the goal is to find the **best solution** from a set of possible solutions.

In Hill Climbing, the algorithm starts with an initial solution and then iteratively makes small changes to it in order to improve the solution. These changes are based on a **heuristic function** that evaluates the quality of the solution. The algorithm continues to make these small changes until it reaches a **local maximum**, meaning that no further improvement can be made with the current set of moves.

There are several variations of Hill Climbing, including steepest ascent Hill Climbing, first-choice Hill Climbing, and simulated annealing. In steepest ascent Hill Climbing, the algorithm evaluates all the possible moves from the current solution and selects the one that leads to the **best improvement**. In first-choice Hill Climbing, the algorithm randomly selects a move and accepts it if it leads to an improvement, regardless of whether it is the best move.

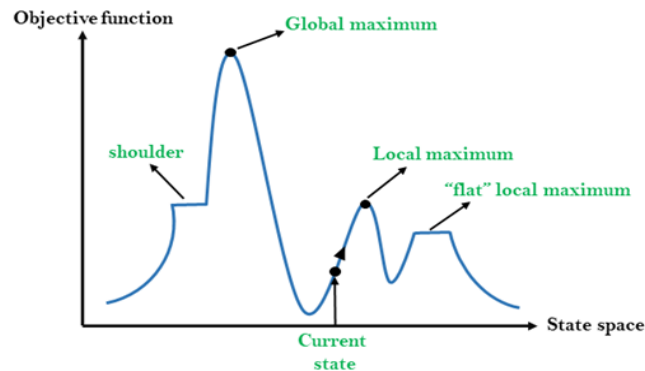
Hill Climbing can be useful in a variety of optimization problems, such as scheduling, route planning, and resource allocation. However, it has some limitations, such as the tendency to get **stuck in local maxima** and the **lack of diversity** in the search space. Therefore, it is often combined with other optimization techniques, such as genetic algorithms or simulated annealing, to overcome these limitations and improve the search results.

Variant of generating and test algorithm

It is a variant of generating and testing algorithms. The generate and test algorithm is as follows :

1. Generate possible solutions.
2. Test to see if this is the expected solution.
3. If the solution has been found quit else go to step 1.

Hence we call Hill climbing a variant of generating and test algorithm as it takes the **feedback** from the test procedure. Then this feedback is utilized by the generator in deciding the next move in the search space.



- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
- **Current state:** It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the same value.
- **Shoulder:** It is a plateau region which has an uphill edge.

Algorithm for Simple Hill climbing

1. Evaluate the **initial state**. If it is a goal state then stop and return success. Otherwise, make the initial state as the current state.
2. Loop until the solution state is found or there are no new operators present which can be applied to the current state.
3. Select a state that has not been yet applied to the **current state** and apply it to produce a new state.
4. Perform these to evaluate the new state.
5. If the current state is a goal state, then stop and return **success**.
6. If it is better than the current state, then make it the current state and proceed further.
7. If it is not better than the current state, then continue in the loop until a solution is found.
8. **Exit** from the function.

Advantages of Hill Climbing algorithm:

1. Hill Climbing is a simple and **intuitive** algorithm that is easy to understand and implement.
2. It can be used in a wide variety of optimization problems, including those with a larger search space and complex constraints.
3. Hill Climbing is often very efficient in finding **local optima**, making it a good choice for problems where a good solution is needed quickly.
4. The algorithm can be easily modified and extended to include additional **heuristics** or constraints.

PROCEDURE

1. Represent the problem of **6 people** travelling to the same destination city Rome from different source cities and vice versa. Specify all the cities as annotations to the people travelling from there.

```
Hill Climb Optimization Algorithm - flight schedule

Representing the problem

people = [('Lisbon', 'LIS'),
          ('Madrid', 'MAD'),
          ('Paris', 'CDG'),
          ('Dublin', 'DUB'),
          ('Brussels', 'BRU'),
          ('London', 'LHR')]

people[3]
('Dublin', 'DUB')

destiny = 'FCO' # Rome

flights = {('BRU', 'FCO'): ['15:44', '18:55', 382]}

flights
{('BRU', 'FCO'): ['15:44', '18:55', 382]}

flights[('BRU', 'FCO')]
['15:44', '18:55', 382]

flights[('BRU', 'FCO')][0], flights[('BRU', 'FCO')][1], flights[('BRU', 'FCO')][2]
('15:44', '18:55', 382)
```

2. Import the dataset **'flights.txt'**. Perform data preprocessing on the dataset so that we can get the origin, destination, departure time, arrival time and the price of each of the flights.

```
flights = {}
for row in open('/content/flights.txt'):
    #print(row)
    #print(row.split(','))
    origin, destination, depart, arrival, price = row.split(',')
    #print(origin, destination, depart, arrival, price)
    flights.setdefault((origin, destination), [])
    flights[(origin, destination)].append((depart, arrival, int(price)))

flights
{('15:34', '18:11', 326),
 ('17:07', '20:04', 291),
 ('18:23', '21:35', 134),
 ('19:53', '22:21', 173),
 ('FCO', 'DUB'): [('6:39', '8:09', 86),
 ('8:23', '10:28', 149),
 ('9:58', '11:18', 130),
 ('10:33', '12:03', 74),
 ('12:08', '14:05', 142),
 ('13:39', '15:30', 74),
 ('15:25', '16:58', 62),
 ('17:03', '18:03', 103),
 ('18:24', '20:49', 124),
 ('19:58', '21:23', 142)],
 ('DUB', 'FCO'): [('6:17', '8:26', 89),
 ('8:04', '10:11', 95)]}
```

```

flights[('LIS', 'FCO')]
[('6:11', '8:31', 249),
 ('7:39', '10:24', 219),
 ('9:15', '12:03', 99),
 ('11:08', '13:07', 175),
 ('12:18', '14:56', 172),
 ('13:37', '15:08', 250),
 ('15:03', '16:42', 135),
 ('16:51', '19:09', 147),
 ('18:12', '20:17', 242),
 ('20:05', '22:06', 261)]

[11] flights[('FCO', 'LIS')]
[('6:19', '8:13', 239),
 ('8:04', '10:59', 136),
 ('9:31', '11:43', 210),
 ('11:07', '13:24', 171),
 ('12:31', '14:02', 234),
 ('14:05', '15:47', 226),
 ('15:07', '17:21', 129),
 ('16:35', '18:56', 144),
 ('18:25', '20:34', 205),
 ('20:05', '21:44', 172)]

```

```

[12] flights[('MAD', 'FCO')]
[('6:05', '8:32', 174),
 ('8:25', '10:34', 157),
 ('9:42', '11:32', 169),
 ('11:01', '12:39', 260),
 ('12:44', '14:17', 134),
 ('14:22', '16:32', 126),
 ('15:58', '18:40', 173),
 ('16:43', '19:00', 246),
 ('18:48', '21:45', 246),
 ('19:50', '22:24', 269)]

[13] flights[('FCO', 'MAD')]
[('6:03', '8:43', 219),
 ('7:50', '10:08', 164),
 ('9:11', '10:42', 172),
 ('10:33', '13:11', 132),
 ('12:08', '14:47', 231),
 ('14:19', '17:09', 190),
 ('15:04', '17:23', 189),
 ('17:06', '20:00', 95),
 ('18:33', '20:22', 143),
 ('19:32', '21:25', 160)]

```

3. Create a function **'print_schedule'** to create a random schedule of 3 journey flights and 3 return flights with their total price.

```

[14] schedule = [0,2, 3,2, 5,3, 6,3, 1,4, 5,3]
len(schedule)
12

len(schedule) // 2
6

```

```

def print_schedule(schedule):
    flight_id = -1
    total_price = 0
    for i in range(len(schedule) // 2):
        name = people[i][0]
        origin = people[i][1]
        flight_id += 1
        go = flights[(origin, destiny)][schedule[flight_id]]
        total_price += go[2]
        flight_id += 1
        ret = flights[(destiny, origin)][schedule[flight_id]]
        total_price += ret[2]

        print('%10s%10s %5s-%5s %3s %5s-%5s %3s' % (name, origin, go[0], go[1], go[2],
                                                    ret[0], ret[1], ret[2]))

    #print(name, origin, go)
    print('Total price: ', total_price)

```

```

0s ✓ print_schedule(schedule)

Lisbon LIS 6:11- 8:31 249 9:31-11:43 210
Madrid MAD 11:01-12:39 260 9:11-10:42 172
Paris CDG 14:01-17:24 338 11:08-14:38 262
Dublin DUB 15:27-17:18 151 10:33-12:03 74
Brussels BRU 7:53-11:37 433 12:20-16:34 500
London LHR 13:40-15:38 137 10:32-13:16 139
Total price: 2925

+ Code + Text

[18] 10**12

1000000000000

```

4. To optimize this whole process and find a schedule of the cheapest flights, we need to apply the hill climbing algorithm to this problem.
5. Create a function 'fitness_function'.

```

Fitness function

0s ✓ def fitness_function(schedule):
    flight_id = -1
    total_price = 0
    for i in range(len(schedule) // 2):
        name = people[i][0]
        origin = people[i][1]
        flight_id += 1
        go = flights[(origin, destiny)][schedule[flight_id]]
        total_price += go[2]
        flight_id += 1
        ret = flights[(destiny, origin)][schedule[flight_id]]
        total_price += ret[2]

    return total_price

[20] schedule = [1,7, 3,1, 1,1, 6,3, 2,4, 5,3]
fitness_function(schedule)

2619

```

6. Pip install the library 'mlrose'.

```

0s ✓ [21] # UPDATE JUNE/2024
# !pip install mlrose
!git clone https://github.com/gkhayes/mlrose.git
!pip install mlrose/

Cloning into 'mlrose'...
remote: Enumerating objects: 853, done.
remote: Counting objects: 100% (253/253), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 853 (delta 229), reused 224 (delta 224), pack-reused 600 (from 1)
Receiving objects: 100% (853/853), 409.15 KiB | 10.49 MiB/s, done.
Resolving deltas: 100% (553/553), done.
Processing ./mlrose
  Preparing metadata (setup.py) ... done
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from mlrose==1.3.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from mlrose==1.3.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from mlrose==1.3.0)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from mlrose==1.3.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from mlrose==1.3.0)
Building wheels for collected packages: mlrose
  Building wheel for mlrose (setup.py) ... done
  Created wheel for mlrose: filename=mlrose-1.3.0-py3-none-any.whl size=27706 sha256=13742a8a756
  Stored in directory: /tmp/pip-ephem-wheel-cache-coggfpe0/wheels/50/ac/56/a248001dcf63344fbef99
Successfully built mlrose
Installing collected packages: mlrose
Successfully installed mlrose-1.3.0

```

7. Import the following libraries :-
 - a) **MLrose** - which is used to apply optimizations and search algorithms to different optimization problems.
 - b) **Six** - which is required to support codebases that work on both Python 2 and Python 3.
 - c) **Sys** - which provides various functions and variables that are used to manipulate different parts of the Python runtime environment.

```

[22] import six
import sys
sys.modules['sklearn.externals.six'] = six
import mlrose

[23] fitness = mlrose.CustomFitness(fitness_function)

problem = mlrose.DiscreteOpt(length = 12, fitness_fn=fitness,
                             maximize = False, max_val = 10) # 0 - 9

```

OUTPUT

1. Apply the Hill climbing algorithm to find the **best solution** price for the flights.

```

Hill climb

[25] best_solution, best_price = mlrose.hill_climb(problem)
best_solution, best_price

(array([2, 6, 5, 7, 8, 1, 3, 6, 0, 2, 9, 1]), 1566.0)

print_schedule(best_solution)

Lisbon LIS 9:15-12:03 99 15:07-17:21 129
Madrid MAD 14:22-16:32 126 17:06-20:00 95
Paris CDG 18:23-21:35 134 8:23-11:07 143
Dublin DUB 11:16-13:29 83 15:25-16:58 62
Brussels BRU 6:12-10:22 230 9:49-13:51 229
London LHR 20:30-23:11 114 8:19-11:16 122
Total price: 1566

```

2. Run all the cells to make sure there are no errors.